CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-6

Problem Definition: -

• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.

• Replace the GitHub repo data from Practical 3 with task data from your own backend.

• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.

• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.

• Handle loading and error states for every API interaction, not just the initial fetch.

Screenshots:-

Server.js:

p6 > backend > JS server.js > ...

```
const express = require('express');
const mongoose = require('mongoose');
require('dotenv').config();

const Task = require('./models/Task');

const cors = require('cors');

const app = express();

app.use(cors());
app.use(express.json());

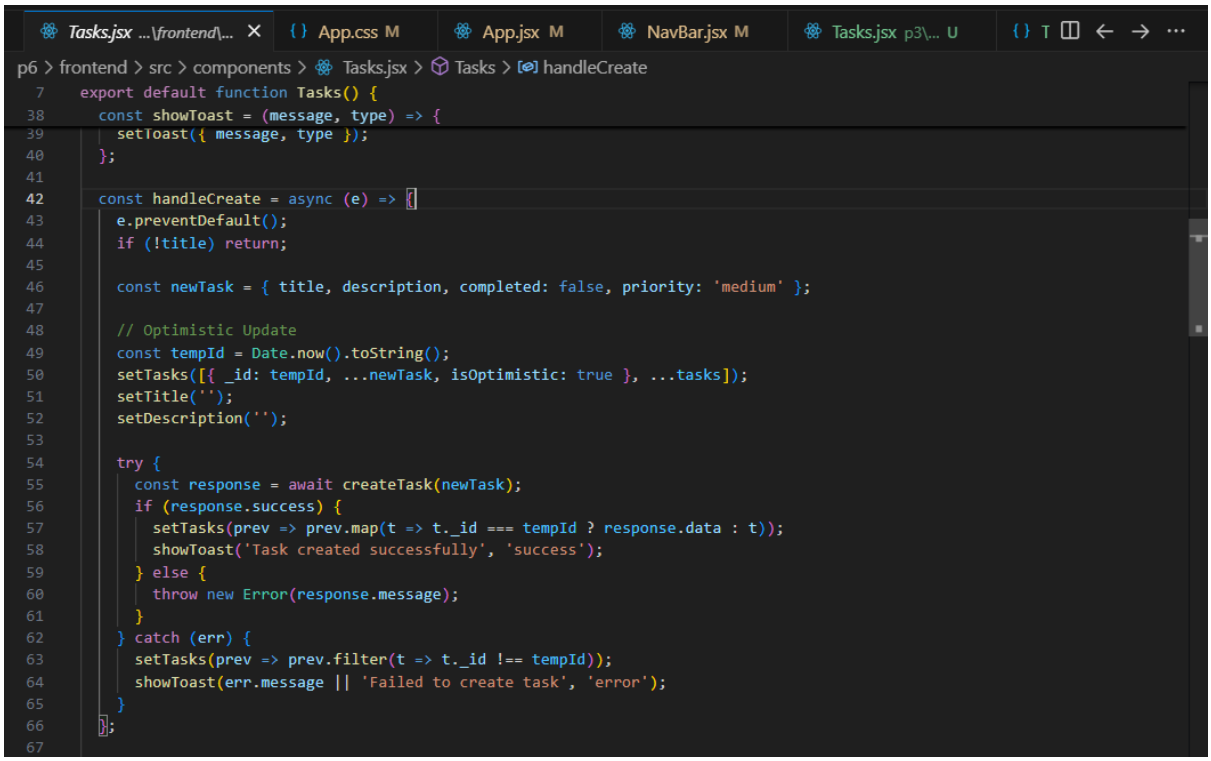
// Request logger middleware
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

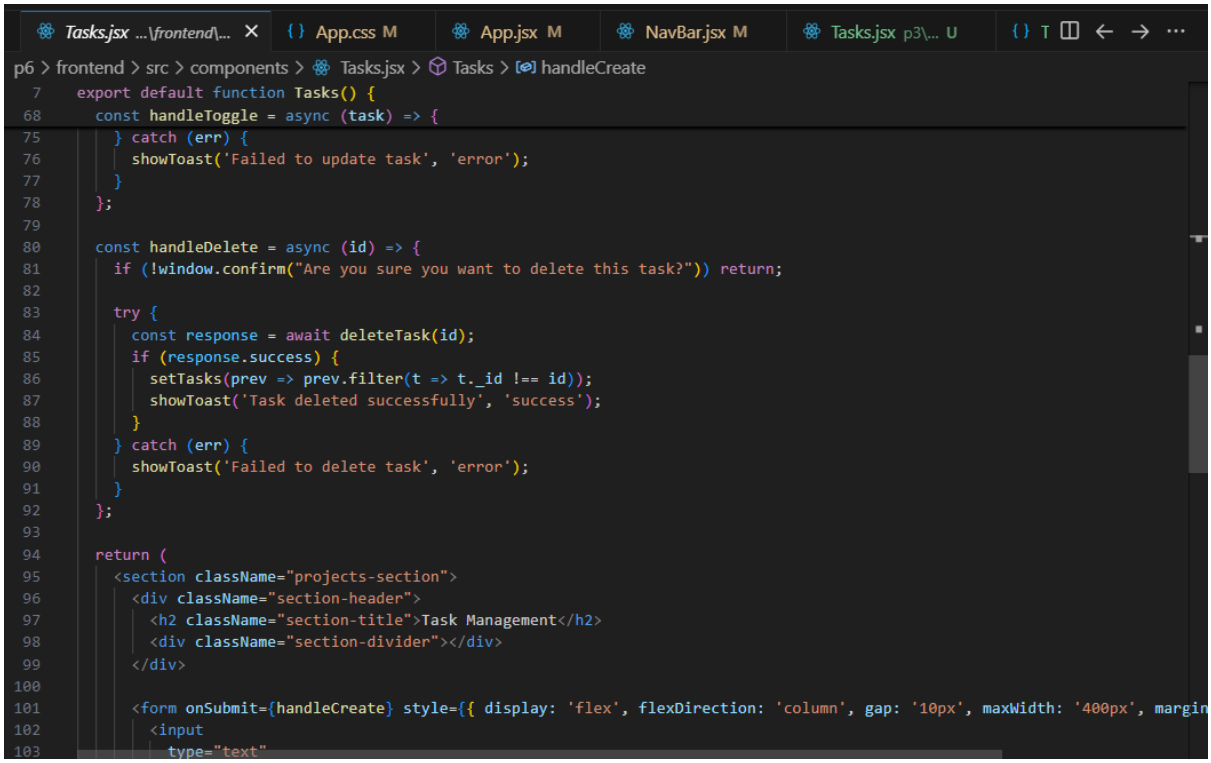
// Content-Type validator for POST/PUT
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
        message: 'Content-Type header must be application/json for POST and PUT requests'
      });
    }
  }
```

Api.js:



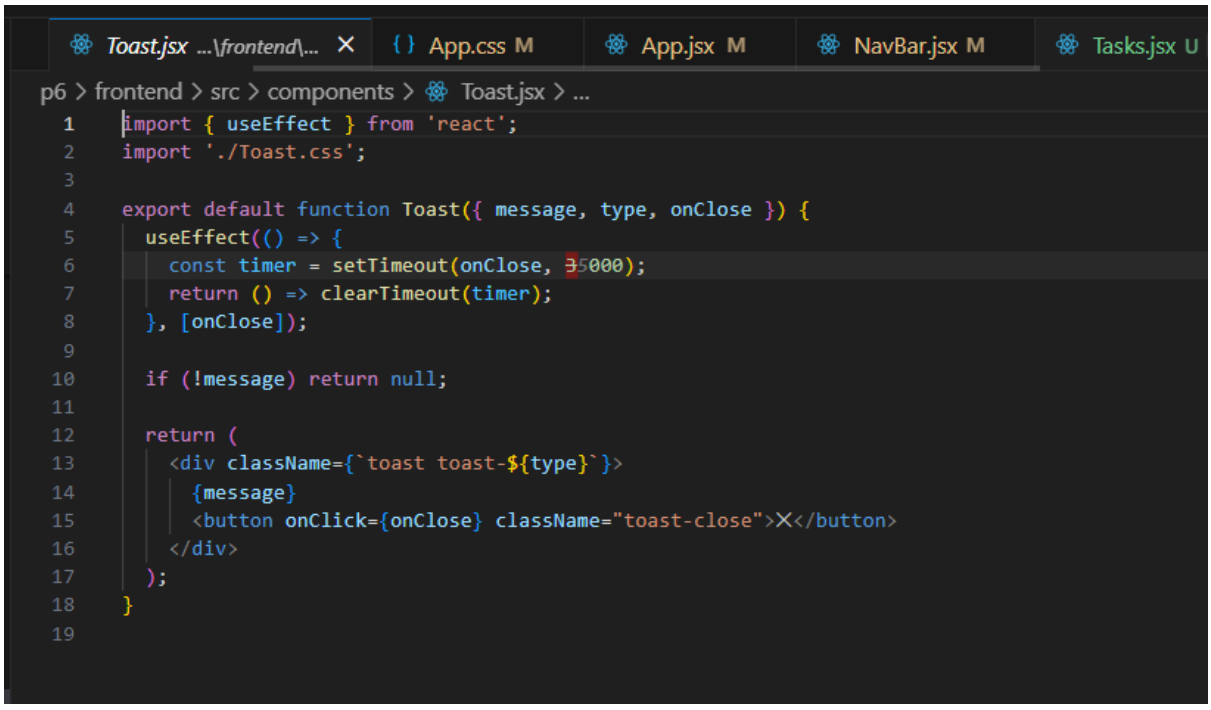
Task.jsx:





```
Tasks.jsx ...\frontend\... | App.css M | App.jsx M | NavBar.jsx M | Tasks.jsx p3\... U
p6 > frontend > src > components > Tasks.jsx > Tasks > handleCreate
 7   export default function Tasks() {
68     const handleToggle = async (task) => {
75       } catch (err) {
76         showToast('Failed to update task', 'error');
77       }
78     };
79
80     const handleDelete = async (id) => {
81       if (!window.confirm("Are you sure you want to delete this task?")) return;
82
83       try {
84         const response = await deleteTask(id);
85         if (response.success) {
86           setTasks(prev => prev.filter(t => t._id !== id));
87           showToast('Task deleted successfully', 'success');
88         }
89       } catch (err) {
90         showToast('Failed to delete task', 'error');
91       }
92     };
93
94     return (
95       <section className="projects-section">
96         <div className="section-header">
97           <h2 className="section-title">Task Management</h2>
98           <div className="section-divider"></div>
99         </div>
100
101        <form onSubmit={handleCreate} style={{ display: 'flex', flexDirection: 'column', gap: '10px', maxWidth: '400px', margin
102          <input
103            type="text"
```

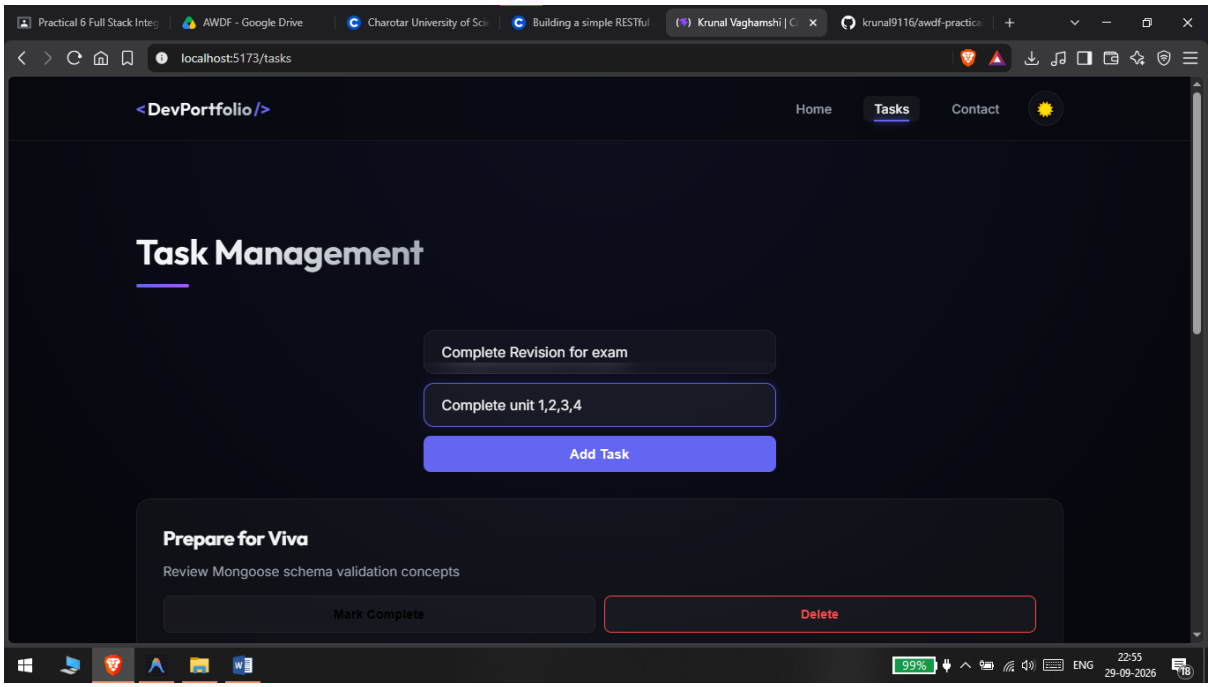
Toast.jsx:



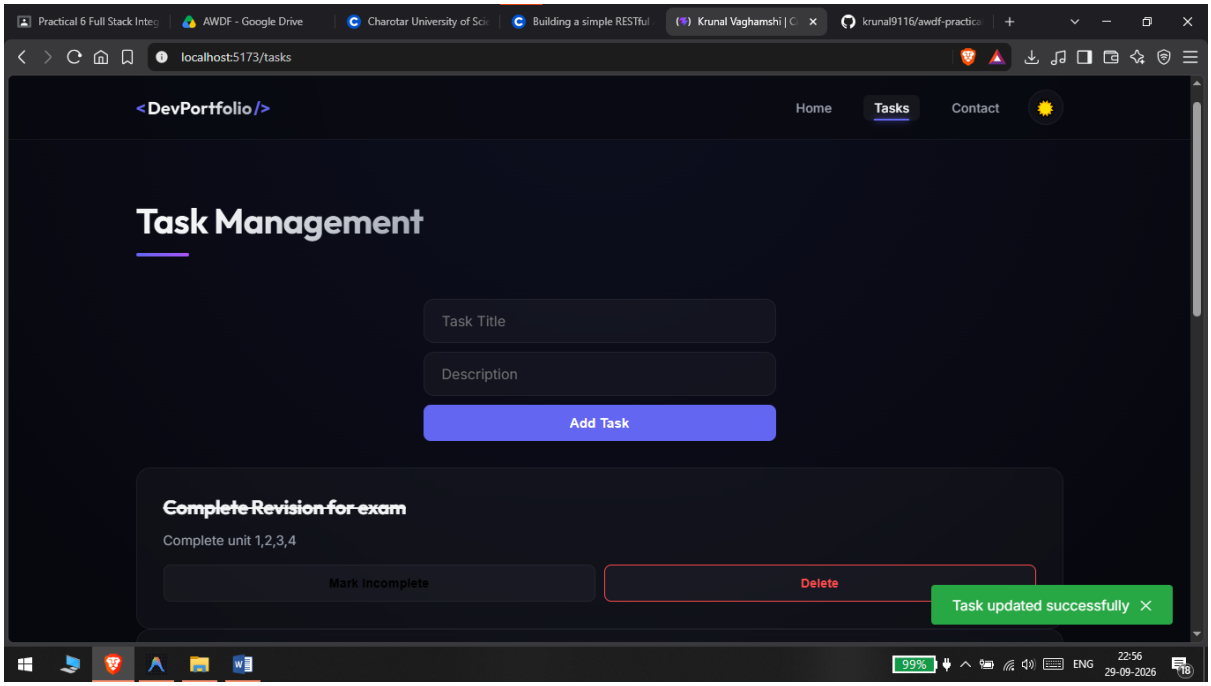
```
Toast.jsx ...\frontend\... | App.css M | App.jsx M | NavBar.jsx M | Tasks.jsx U
p6 > frontend > src > components > Toast.jsx > ...
1   import { useEffect } from 'react';
2   import './Toast.css';
3
4   export default function Toast({ message, type, onClose }) {
5     useEffect(() => {
6       const timer = setTimeout(onClose, 5000);
7       return () => clearTimeout(timer);
8     }, [onClose]);
9
10    if (!message) return null;
11
12    return (
13      <div className={`toast toast-${type}`}>
14        {message}
15        <button onClick={onClose} className="toast-close">X</button>
16      </div>
17    );
18  }
19
```

Output:

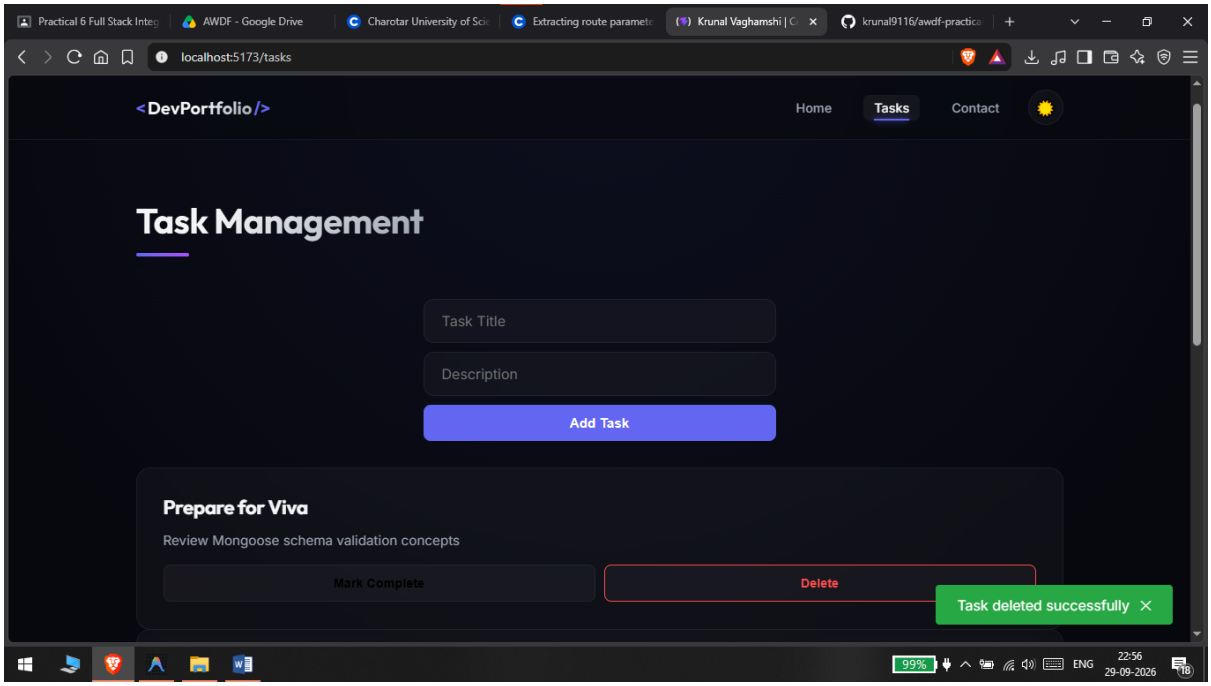
Task creation:



Updating the task to Completed:



Deleting the Task:



Github Link:- [Practical-6 Repository (AWDF)]